FASE 2: DISEÑADOR DE SOLUCIONES (6-18 MESES)

Rol funcional: Automation Engineer con visión de arquitectura **Objetivo:** €3.5-5k/mes + sistemas complejos en producción **Identidad profesional:** "Diseño e implemento soluciones de automatización + IA para empresas"

📊 DISTRIBUCIÓN DE TIEMPO

Total: 25-30 horas/semana

Actividad	Horas/semana	%
Trabajo remunerado	12-15h	45%
Backend + APIs propias	6-8h	25%
Compliance + Governance	2-3h	10%
Cloud + Deploy	2-3h	10%
Networking + Posicionamiento	2-3h	10%

🎓 COMPETENCIAS TÉCNICAS - FASE 2

1. PYTHON INTERMEDIO-AVANZADO (25% esfuerzo)

Ahora SÍ profundizas en Python:

python

✅ POO (Programación Orientada a Objetos):

- Classes, methods, attributes
- Inheritance, composition
- Abstract classes (ABC)
- Encapsulation básica

✅ Async/await:

- Qué es concurrencia
- asyncio básico
- Cuándo usar async

✅ Type hints:

- Anotaciones de tipos
- mypy básico

✅ Context managers:

- with statement
- __enter__, __exit__

✅ Decorators (entender, no crear):

- @property
- @staticmethod, @classmethod
- Decorators de frameworks

✅ Error handling profesional:

- Custom exceptions
- Logging estructurado
- Traceback analysis

✅ Testing básico:

- pytest
- Unit tests simples
- Mocking básico

Recursos:

- "Fluent Python" (caps selectos, no todo)
- Real Python (artículos específicos)
- Python Type Checking Guide

Proyecto: Script complejo (300+ líneas) con:

- Múltiples clases
- Error handling robusto
- Logging
- Type hints
- Tests básicos

2. BACKEND CON FASTAPI (30% esfuerzo)

Por qué FastAPI:

- Moderno, rápido, type-safe
- Auto-documentación (Swagger)
- Async nativo
- Pythonic

Competencias:

python

✅ FastAPI básico:

- Routes (GET, POST, PUT, DELETE)

- Path parameters
- Query parameters
- Request body (Pydantic models)
- Response models
- Status codes

✅ Autenticación:

- JWT tokens
- OAuth2 básico
- Dependency injection
- Protected routes

✅ Database:

- SQLAlchemy (ORM)
- Models y relationships
- Migrations con Alembic
- Queries eficientes

✅ Error handling:

- HTTPException
- Custom exception handlers
- Validation errors

✅ Middleware:

- CORS

- Authentication
- Logging

✅ Background tasks:

- Async tasks
- Celery conceptual (no deep)

✅ API design:

- RESTful conventions
- Versioning
- Pagination
- Filtering

Recursos:

- FastAPI docs (excelentes)
- "Building APIs with FastAPI" (YouTube playlists)
- FastAPI + PostgreSQL tutorial

Proyecto: API completa con:

- CRUD operations
- Autenticación JWT
- PostgreSQL backend
- Tests
- Documentación Swagger
- Deploy en cloud

3. BASES DE DATOS AVANZADO (15% esfuerzo)

sql

✅ PostgreSQL avanzado:

- Índices (B-tree, Hash, GiST)
- Query optimization

- EXPLAIN ANALYZE
- Transactions y ACID
- Constraints avanzados
- Views y Materialized views
- Stored procedures básicos

✅ Redis:
- Caching patterns
- Session storage
- Rate limiting
- Pub/Sub básico
- Data structures (strings, hashes, lists, sets)

✅ Design patterns:
- Cuándo SQL vs NoSQL
- Normalización vs denormalización
- Sharding conceptual
- Replication conceptual

```

**Proyecto:**

Sistema con:

- PostgreSQL (data principal)
- Redis (caching + sessions)
- Métricas de performance (antes/después de índices)
```

---

4. CLOUD INTERMEDIO (10% esfuerzo)

```

✅ Docker:

- Dockerfile bien escrito
- Multi-stage builds
- Docker Compose
- Volumes y networks
- Best practices

✅ AWS intermedio:

- Lambda (functions, triggers, layers)
- RDS (managed databases)
- S3 (storage, lifecycle policies)
- CloudWatch (logs, metrics, alarms)
- IAM (users, roles, policies)
- API Gateway básico

✅ CI/CD:

- GitHub Actions
- Automated testing
- Automated deployment
- Environment variables
```

✅ Monitoring:

- Logs estructurados
- Metrics (latency, errors, throughput)
- Alertas básicas

```

**Proyecto:**

App desplegada con:

- Docker containerizada
- Deploy automatizado (GitHub Actions)
- RDS database
- Logs en CloudWatch
- Health check endpoint

---

### **5. SISTEMAS DISTRIBUIDOS BÁSICO (10% esfuerzo)**

**Conceptual + hands-on básico:**

```

✅ Message queues:

- RabbitMQ O AWS SQS
- Producer/Consumer pattern
- Dead letter queues

- Retry strategies

✅ Webhooks avanzado:

- Idempotencia
- Retry con exponential backoff
- Signature verification
- Webhook testing

✅ Async processing:

- Cuándo usar async vs sync
- Task queues
- Background jobs
- Scheduling

✅ Rate limiting:

- Token bucket algorithm
- Fixed window vs sliding window
- Implementación con Redis

```

**Proyecto:**

Sistema asíncrono:

- API recibe request
- Encola tarea (RabbitMQ/SQS)
- Worker procesa
```

- Callback cuando termina

- Retry si falla

---

6. COMPLIANCE & GOVERNANCE ⭐ (10% esfuerzo)

CRÍTICO PARA MERCADO EUROPEO

```

✅ GDPR aplicado:

  - Data mapping (qué datos guardas, dónde)

  - Consent management

  - Right to deletion (implementación técnica)

  - Data portability

  - Breach notification procedures

✅ Data governance:

  - Data lineage (de dónde viene, a dónde va)

  - Access control (RBAC básico)

  - Audit trails (quién accedió a qué, cuándo)

  - Data retention policies

  - Data masking (entornos no-prod)

✅ Security basics:

  - Input validation
```

- SQL injection prevention
- XSS prevention
- HTTPS everywhere
- Secrets management

✅ ISO 27001 conceptual:
- Qué es
- Controles principales
- Information Security Management System (ISMS)

```

**Recursos:**

- GDPR Developer Guide
- OWASP Top 10
- "Security for Developers" (curso)

**Proyecto obligatorio:**

Audita sistema existente:

- Identifica PII
- Mapea flujo de datos
- Implementa audit trail
- Añade data deletion endpoint
- Documenta compliance measures

---
```

🏗️ PROYECTOS OBLIGATORIOS FASE 2

PROYECTO 1: API + DB + Deploy + Compliance

Full-stack system con todo lo aprendido:

```
Spec técnico:
- FastAPI backend
- PostgreSQL database
- Autenticación JWT
- GDPR compliant:
  * Data mapping documentado
  * Audit trail (logs de accesos)
  * Delete endpoint (right to be forgotten)
  * Data export endpoint (portability)
- Deploy en AWS (EC2 o Lambda + RDS)
- CI/CD (GitHub Actions)
- Monitoring (CloudWatch)
- Tests (pytest, coverage 70%+)
```

Ejemplo real: "Task Management API"

```
Features:
```

- Users, Projects, Tasks

- Roles (admin, member)

- CRUD operations

- Search y filtering

- Audit trail de cambios

- Export personal data

- Delete account completo

GDPR compliance:

- Consents tracked

- Data mapped

- Retention policy (tasks deleted after 2 years)

- Right to deletion implemented

```

**Documentación obligatoria:**

- API docs (Swagger)

- Architecture diagram

- ADR: decisiones clave

- GDPR compliance doc

- Deployment guide

**Tiempo estimado:** 4-6 semanas

---
```

PROYECTO 2: Sistema ETL con Governance

Data pipeline productivo:

```

Pipeline:

1. Extract:

   - Scraping web O

   - API externa O

   - Multiple data sources


2. Transform:

   - Python processing

   - Data cleaning

   - Enrichment con IA

   - Data masking (PII)


3. Load:

   - PostgreSQL (structured)

   - S3 (raw data backup)


4. Monitoring:

   - Success/failure rates

   - Data quality metrics

   - Cost tracking
```

```

**Governance:**

```

✅ Data lineage documented

✅ PII identified y masked en non-prod

✅ Access control (solo ciertos users pueden ver PII)

✅ Audit trail (quién ejecutó qué, cuándo)

✅ Retention policy (raw data deleted after 90 days)

```

**Ejemplo: "Competitor Monitoring System"**

```

Extrae:

- Precios de 10 competidores (scraping)

- Reviews de G2/Capterra (API)

Transforma:

- Normaliza precios (monedas)

- Sentiment analysis con IA

- Detecta tendencias

Carga:

- PostgreSQL (structured data)

- Dashboard (Metabase o similar)

Governance:

- Emails de competidores son PII → masked

- Audit de quién accede a qué datos

- Data retention 180 días

```

**Documentación:**

- Data flow diagram

- ADR: por qué esta arquitectura

- Data governance policy

- Cost analysis (€/mes para correr)

---

### **PROYECTO 3: Cost Optimization Challenge ⭐**

**Este proyecto demuestra valor económico real:**

```

Objetivo:

Toma sistema existente (tuyo o de cliente) y reduce costos 40%+

sin degradar calidad significativamente.

```

**Metodología:**
```

1. Baseline (semana 1):

```

Documenta estado actual:

- Costo mensual total

- Desglose por componente:

  * API calls (OpenAI, Claude, etc.)

  * Cloud resources (EC2, Lambda, RDS, S3)

  * Third-party services

- Volumen de operaciones

- Métricas de calidad (latency, accuracy)

```

2. Análisis (semana 2):

```

Identifica oportunidades:

- ¿Qué se puede cachear?

- ¿Qué queries se pueden optimizar?

- ¿Qué modelos IA se pueden reemplazar por más baratos?

- ¿Qué procesos se pueden hacer batch vs real-time?

- ¿Qué recursos cloud están sobre-provisionados?

```

3. Implementación (semanas 3-4):

```
```

Implementa mejoras:

- Caching strategy (Redis)

- Query optimization (índices, rewrite)

- Model tiering:

  * GPT-4 → GPT-3.5 para queries simples

  * Anthropic Claude Opus → Sonnet cuando suficiente

  * Local model para casos muy simples

- Batch processing donde sea posible

- Auto-scaling en lugar de always-on

```

**4. Validación (semana 5):**

```

Compara resultados:

- Costo nuevo vs baseline

- % reducción

- Impacto en calidad (si hay)

- Trade-offs aceptados

Documentación (crítica):

markdown

Cost Optimization Case Study

Baseline

- Sistema: [descripción]

- Costo mensual: €2,400

- Operaciones: 50k requests/mes

- Latency p95: 800ms

Análisis

- 60% costos en OpenAI GPT-4

- 25% en AWS EC2 (over-provisioned)

- 15% en RDS

Cambios Implementados

1. Model tiering:

   - Clasificador determina complejidad

   - 70% queries → GPT-3.5 (suficiente)

   - 30% queries → GPT-4 (necesario)

   - Savings: €720/mes

2. Caching agresivo:

   - Redis para queries repetidas

   - 40% hit rate

   - Savings: €280/mes

3. Auto-scaling EC2:

   - Scale down en off-hours

   - Savings: €300/mes

Resultados

- Costo nuevo: €1,100/mes

- Reducción: 54% (€1,300/mes)

- Latency p95: 750ms (mejora!)

- Accuracy: sin cambio significativo

Trade-offs

- Complejidad añadida (classifier + routing)

- Mantenimiento de cache

- Riesgo: si clasificador falla

ROI

- Savings anual: €15,600

- Tiempo implementación: 4 semanas

- Payback: inmediato

```

**ESTE PROYECTO ES TU PORTFOLIO DE ARQUITECTO.**

---

## 💰 MONETIZACIÓN FASE 2

**Objetivo:** €3.5-5k/mes consistente

### **EVOLUCIÓN DE SERVICIOS**
```

Ya no vendes automatizaciones simples.

Vendes sistemas.

Servicios Fase 2:

```

1. "Soluciones de Automatización + IA Custom"

   - Sistemas híbridos (no-code + código cuando necesario)

   - Integración de múltiples plataformas

   - IA integrada inteligentemente

   - GDPR compliant


   Pricing: €2.5-5k por sistema

   Timeline: 3-6 semanas


2. "Optimización de Costos de Sistemas IA"

   - Audit actual

   - Optimizaciones implementadas

   - Garantía de 30%+ reducción


   Pricing: 30% de savings primer año

   O: €2-4k flat fee


3. "Consultoría de Arquitectura"

   - System design
```

- Tech stack selection

- Compliance review

- Cost optimization strategy

Pricing: €800-1.5k/día

O: €100-150/hora

4. "Migraciones Make → n8n (managed)"

- Migración de workflows

- Setup infrastructure

- Training del equipo

Pricing: €3-6k depending complejidad

```

### **POSICIONAMIENTO**

**LinkedIn:**

```

Solutions Engineer | Automation + IA

Ayudo a empresas a reducir costos operativos 40%+

con sistemas IA + automatización GDPR-compliant

- Python | FastAPI | n8n | PostgreSQL | AWS

- Especializado en optimización de costos IA

- 15+ sistemas en producción

📍 Rovereto, Italia (remoto UE)

```

**Portfolio site necesario:**

```

tudominio.com

- Sobre mí

- Servicios

- Casos de estudio (3-5)

- Blog (artículos técnicos)

- Contacto

CANALES DE ADQUISICIÓN

1. LinkedIn (principal):

- Publica casos de estudio mensualmente
- Comparte aprendizajes técnicos
- Conecta con CTOs, Heads of Engineering
- Participa en discusiones relevantes

2. Referrals (más efectivo):

- Programa de referidos (10-15% fee)
- Pide testimonials a clientes satisfechos
- Casos de estudio con nombre de empresa (con permiso)

3. Networking técnico:

- Meetups (Trento, Verona, Milán)
- Conferencias (habla sobre optimización de costos IA)
- Comunidades online (n8n, FastAPI, etc.)

4. Content marketing:

- Blog posts técnicos

- YouTube tutorials (opcional)
- Newsletter (casos de estudio)

✅ VALIDACIÓN FASE 2 → FASE 3

Criterios para pasar a Fase 3:

Validación Técnica:

- FastAPI: has construido 3+ APIs completas
- Databases: entiendes optimization, índices, transactions
- Docker: puedes containerizar cualquier app
- AWS: has desplegado 5+ sistemas diferentes
- CI/CD: tienes pipelines automatizados funcionando

Validación de Proyectos:

- 2-3 sistemas complejos en producción (clientes reales)
- Al menos 1 cost optimization con resultados documentados
- Sistema compliant con audit documentado
- Portfolio de 10+ ADRs

Validación de Negocio:

- €3.5-5k/mes sostenidos 6 meses
- Al menos 5 clientes satisfechos
- 2+ testimonials/referencias
- Pipeline de prospects (3+ conversaciones activas)

Validación de Posicionamiento:

- Te llaman "engineer" o "architect", no "developer"
- Clientes te consultan antes de decidir (no solo para ejecutar)
- Alguien te pidió opinión arquitectónica no solicitada
- Has rechazado trabajo que no encaja con tu visión

Validación de Mentalidad:

- Documentas decisiones naturalmente
- Piensas en trade-offs automáticamente
- Puedes defender tus decisiones ante stakeholders
- Identificas riesgos de compliance/costos sin pensar

🚨 SEÑALES DE ALERTA FASE 2

🔴 **ALERTA ROJA:**

- Mes 12 y aún construyes solo APIs REST genéricas
- No has reducido costos en ningún sistema

- Clientes te piden "más código" (no diseño)
- €3k/mes estancado 6+ meses

Acción: Estás derivando a backend dev. Re-posiciónate como arquitecto/consultor.

🟡 **ALERTA AMARILLA:**

- Solo trabajo de implementación, nada de consultoría
- No documentas decisiones
- Portafolio es código, no sistemas
- Competencia brutal en tu pricing

Acción: Sube precios. Enfócate en proyectos más complejos.

🟢 **SEÑAL VERDE:**

- Clientes te preguntan "¿debemos hacer X?" (no "¿puedes hacer X?")
- Rechazas trabajo simple (no vale tu tiempo)
- Ingresos por consultoría crecen vs implementación
- Te llaman para diseñar, no solo ejecutar

[redacted]